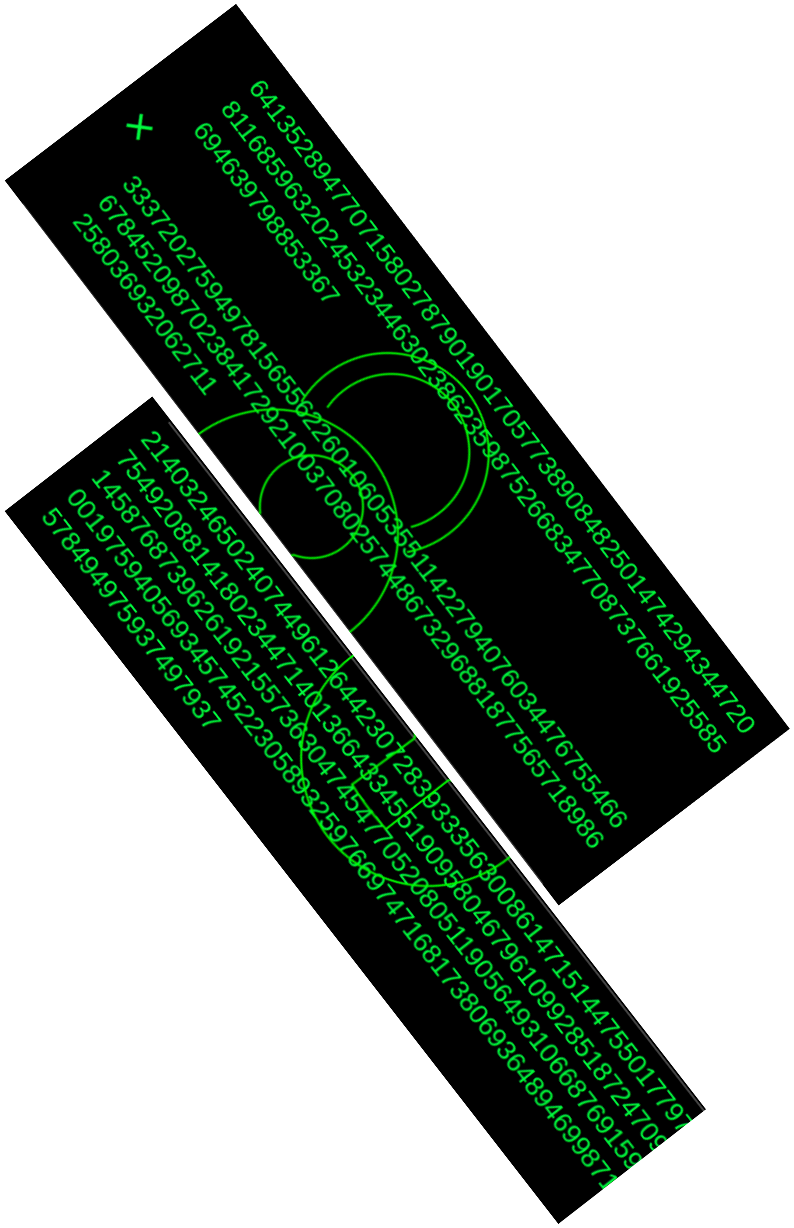


Lecture 4: Quantum Factorization Algorithm

COMP3366 Quantum algorithms & computing architecture

Instructor: Yuxiang Yang

Department of Computer Science, HKU

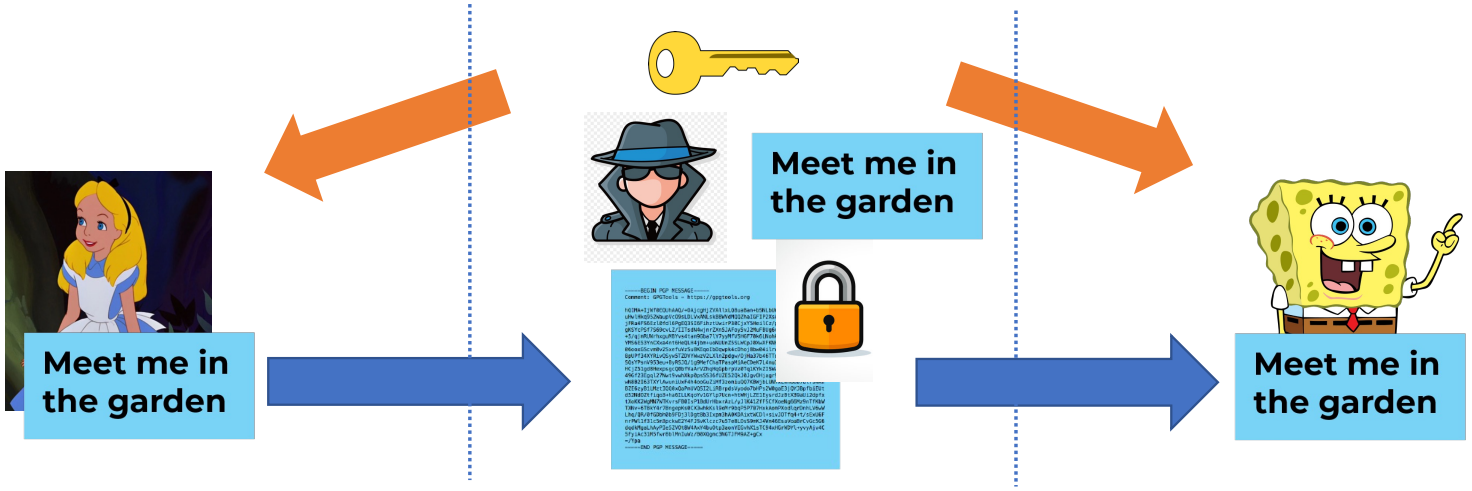
Objectives:

- **[O1] Concepts:** Factorization in cryptography. QFT, QPE, Shor's algorithm.
- **[O2] Problem solving:** Comprehension of QFT and QPE algorithm, Understanding the working principle of Shor's algorithm.
- **[O3] Algorithm design:** Capability of applying QFT, QPE, and phase kickback trick in algorithm design.

Part I:
RSA and Factorization

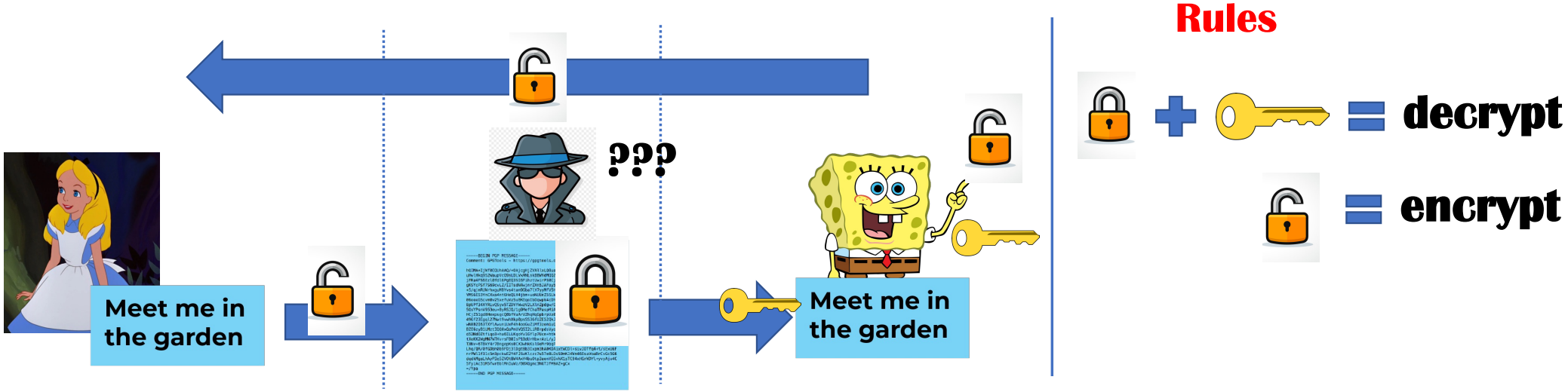
How Alice and Bob could share a secret via cryptography:

Private key system



- The private key system requires a communication channel that is absolutely secure to transfer
- Problem: If a spy gets the message will be given away.
- One could use quantum key distribution (see COMP3357) which, however, is still technically challenging.

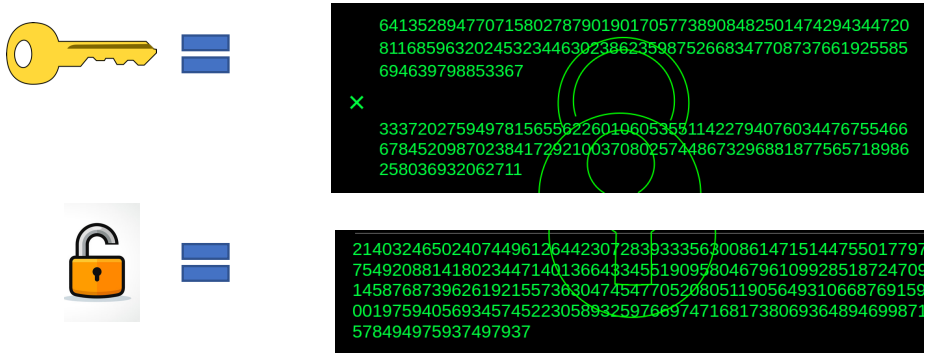
Sharing a secret without requiring a shared key!



RSA (Public key) system

- Bob generates both  and  . He sends  to Alice via a public (i.e., insecure) channel.
- Alice uses  to encrypt her message.
- Even if a spy intercepts the message, without  he cannot decrypt!
- How is the realized in practice?

RSA cryptosystem



- In RSA, Bob generates large number $N = p \cdot q$ with prime p, q
- n is sent to Alice while p and q are held privately by Bob.
- Since factorizing a large number takes forever*, a spy cannot learn p, q even if he gets N.
  - In 2010 (Kleinjung et al.), it took **100+** computers for more than **2 years** to factor a 768-bit number.
- Applications of RSA:



Digital Signature
Market = 1.1B USD (2019)



An RSA SecureID 800 for banks

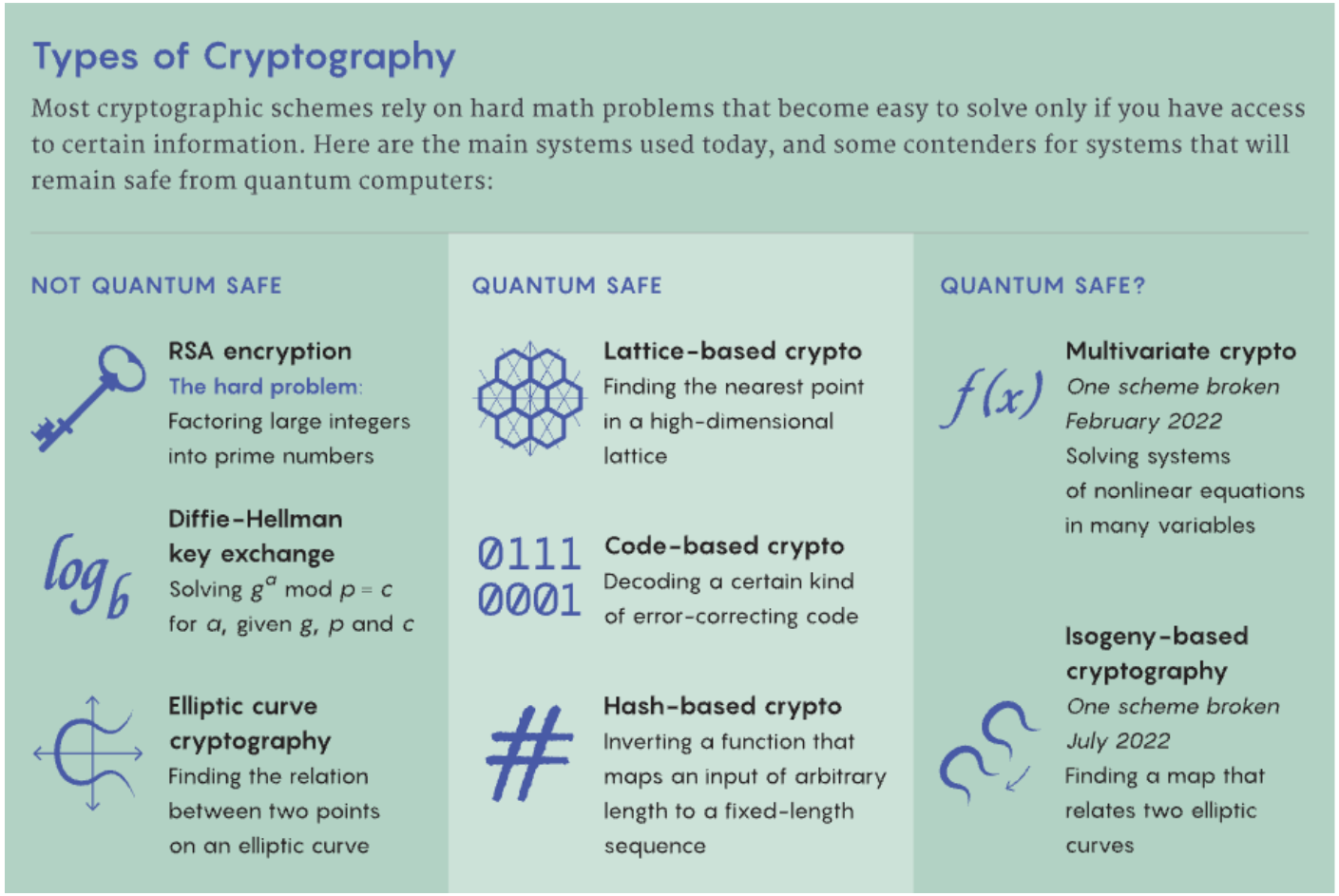
RSA can be cracked by a quantum computer!

- RSA assumes that factorization is computational intractable.
- Shor's algorithm (1995):
  A quantum computer can offer an exponential speedup over existing classical algorithms in factorization!
- In 2010 (Kleinjung et al.),
  it took **100+** computers for more than **2 years** to factor a 768-bit number.
  For a **quantum computer** with **1G(10^9)qubits**, it could take **less than a day**.
- RSA and thus many passwords, bank transfer, secret docs will no longer safe, if we have a full-fledged quantum computer.
  Possible remedies:
  1) Private keys via Q-crypto (BB84 etc.)
  2) Post-quantum crypto



Peter Shor

Quantum computers vs. cryptography



Source: QuantaMagazine

Next: We will unveil the secret of Shor's algorithm

- Step 1 (digital to frequency domain): Learn a quantum circuit named QFT.
- Step 2 (pattern extraction): Use QFT for a key subroutine named QPE.
- Step 3 (factorization to pattern extraction): Give rigorous formulation of the Shor's problem and its oracle.
- Step 4: Discuss detailed construction of Shor's algorithm.



Peter Shor
(when Shor's algorithm was discovered)

Notice: Long (but interesting) journey ahead

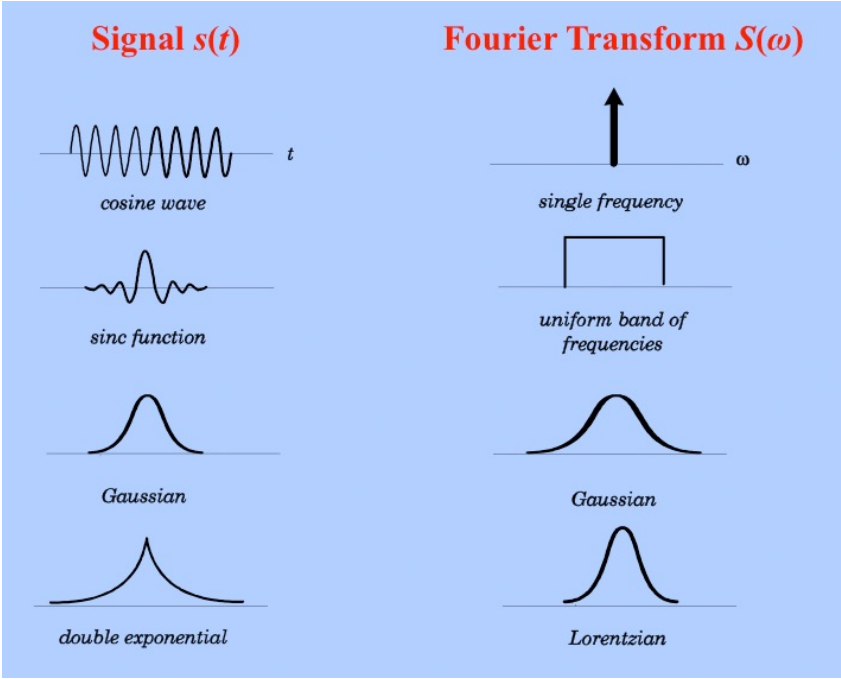


Peter Shor
(present days)

Part II:
Quantum Fourier Transform

Fourier transform

- Fourier transform is the corner stone of modern engineering.
- On the other hand, it is essentially a change of basis:



Quantum Fourier transform (QFT)

- Classical discrete Fourier transform: $x_k \mapsto \sum_j x_j \cdot e^{-\frac{2\pi i k}{d} j}$
- QFT is the quantum generalization of the (classical) discrete Fourier transform, a change of basis from the computational basis to the Fourier basis.
- Fourier basis:
  The Fourier basis $\{|e_k\rangle\}_{k=0}^{d-1}$ on a d-dimensional quantum system is defined as

$$|e_k\rangle := \frac{1}{\sqrt{d}} \sum_{j=0}^{d-1} e^{\frac{(2\pi k j) i}{d}} |j\rangle$$

Exercise:
Show that the Fourier basis is indeed an orthonormal basis.

- Alternatively, we write

$$|e_k\rangle = \frac{1}{\sqrt{d}} \sum_{j=0}^{d-1} \omega^{kj} |j\rangle \qquad \omega := e^{\frac{2\pi i}{d}}$$

Quantum Fourier transform (QFT)

- Fourier basis:
  The Fourier basis $\{|e_j\rangle\}_{j=0}^{d-1}$ on a d-dimensional quantum system is defined as

$$|e_j\rangle := \frac{1}{\sqrt{d}} \sum_{k=0}^{d-1} e^{\frac{(2\pi jk)i}{d}} |k\rangle.$$

- Interpretation: regarding $|e_j\rangle$ as a wave, j is its frequency.
- Quantum Fourier transform:
  QFT transforms the computational basis to the Fourier basis

$$QFT := \sum_{j=0}^{d-1} |e_j\rangle\langle j|$$

QFT for n qubits

- QFT adopts a convenient form when applied on a n-qubit register.
- We often represent the computational basis state $|j_1 j_2 \cdots j_n\rangle$ with $j_1, \ldots, j_n \in \{0,1\}$ by $|j\rangle$ with $j := 2^{n-1} j_1 + \cdots + 2 j_{n-1} + j_n$.

- QFT on a n-qubit register is given by $QFT = \sum_j |e_j\rangle\langle j|$ with

binary

$$|e_j\rangle := \bigotimes_{l=1}^{n} \left(\frac{|0\rangle + e^{2\pi i \cdot (0.j_{n-l+1} \cdots j_n)} |1\rangle}{\sqrt{2}} \right).$$

- Hint: notice that $e^{2\pi i \cdot kj/2^n} = e^{2\pi i \cdot k(0.j_1 j_2 \cdots j_n)}$.

Exercise:
Prove that this is an equivalent definition of $|e_j\rangle$ as the previous one.

- We write $j = 2^{n-1}j_1 + \cdots + 2j_{n-1} + j_n$ and similarly for k.
- Starting from the original definition, we have

$$|e_j\rangle = \frac{1}{\sqrt{2^n}} \sum_{k=0}^{2^n-1} e^{2\pi i \cdot \frac{j}{2^n} \cdot k} |k\rangle$$

$$= \frac{1}{\sqrt{2^n}} \sum_{k=0}^{2^n-1} e^{2\pi i \cdot (0.j_1 j_2 \cdots j_n) \cdot k} |k\rangle$$

$$= \frac{1}{\sqrt{2^n}} \sum_{k_1,k_2,\ldots,k_n=0}^{1} \bigotimes_{l=1}^{n} \left(e^{2\pi i \cdot 0.j_1 j_2 \cdots j_n \cdot (2^{n-l} k_l)} |k_l\rangle \right)$$

$$= \frac{1}{\sqrt{2^n}} \bigotimes_{l=1}^{n} \left(\sum_{k_l=0}^{1} e^{2\pi i \cdot 0.j_1 j_2 \cdots j_n \cdot (2^{n-l} k_l)} |k_l\rangle \right)$$

$$= \bigotimes_{l=1}^{n} \left(\frac{|0\rangle + e^{2\pi i \cdot 0.j_1 j_2 \cdots j_n \cdot 2^{n-l}} |1\rangle}{\sqrt{2}} \right)$$

$$= \bigotimes_{l=1}^{n} \left(\frac{|0\rangle + e^{2\pi i \cdot 0.j_{n-l+1} \cdots j_n} |1\rangle}{\sqrt{2}} \right)$$

$k = \sum_{l=1}^{n} 2^{n-l} k_l$

$|k\rangle = |k_1\rangle \otimes \cdots \otimes |k_n\rangle = \otimes_{l=1}^{n} |k_l\rangle$

Implementing QFT

- Since $QFT = \sum_j |e_j\rangle\langle j|$ with $|e_j\rangle := \otimes_{l=1}^{n} \left(\frac{|0\rangle + e^{2\pi i \cdot (0.j_{n-l+1} \cdots j_n)} |1\rangle}{\sqrt{2}} \right)$ we can implement it by controlled-phase gates and H.

$= \frac{j_{n-l+1}}{2^1} + \cdots + \frac{j_n}{2^l}$

- Phase gates (rotating k-th digit) $R_k = |0\rangle\langle 0| + e^{2\pi i \cdot 2^{-k}} |1\rangle\langle 1|$.
- We can rewrite $|e_j\rangle$ as

for a matrix A, we denote $A^0 = I$ and $A^1 = A$.

$$|e_j\rangle = \otimes_{l=1}^{n} \left(R_l^{j_n} \cdots R_1^{j_{n-l+1}} \boxed{R_1^{j_l} H |j_l\rangle} \right)$$

$= |+\rangle$ regardless of j_l

To cancel the (-1) amplitude of $|1\rangle$, introduced by H when $j_l = 1$.

- The state of the l-th qubit depends on the last l qubits.
- (Failed) Attempt: Starting from $l = 1$, we run on the l-th qubit
  1. Hadamard H,
  2. gate R_1 if $j_l = 1$,
  3. for $k \geq n - l + 1$, R_{k-n+l} conditioned on the k-th qubit being $|1\rangle$.



Exercise: Show that we have $|+\rangle = R_1^a H |a\rangle$ for $a = 0,1$.

- Problems:

  1) 2nd step requires us to "remember" the value of j_l

  2) executing the procedure for $l = n$, the 1st qubit is no longer in $|j_1\rangle$

Implementing QFT

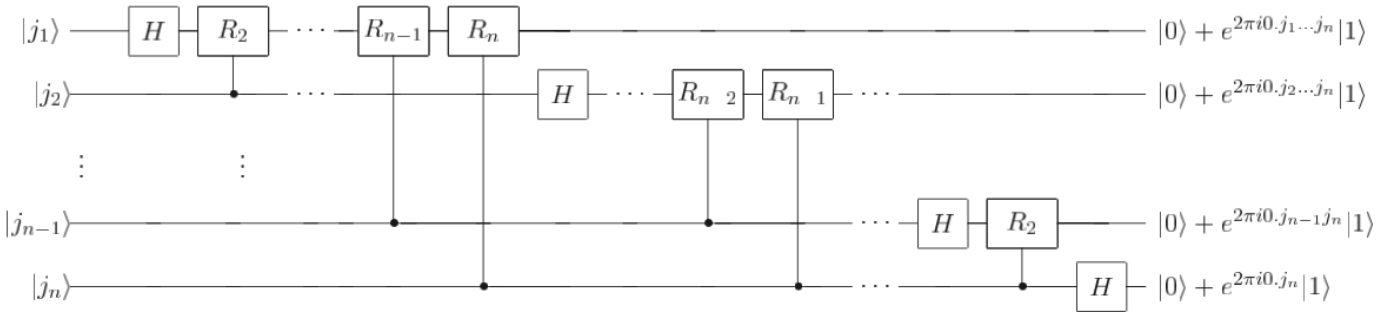
- Solution to the problem:
  We generate the l-th qubit of the output on the $(n-l)$-th qubit of the input, and then swap the qubits.
- We first generate $|e_j\rangle' = \otimes_{l=1}^{n}\left(R_{n-l+1}^{j_n}\cdots R_2^{j_{l+1}}H|j_l\rangle\right)$ and then $|e_j\rangle = \otimes_{l=1}^{n}\left(R_l^{j_n}\cdots R_2^{j_{n-l+2}}R_1^{j_{n-l+1}}R_1^{j_l}H|j_l\rangle\right)$ by swapping the qubits in $|e_j\rangle'$.

Exercise:
Prove that the l-th qubit of $|e_j\rangle$ is identical to the $(n-l+1)$-th qubit of $|e_j\rangle'$.

- To generate $|e_j\rangle'$ when the input is $|j_1\cdots j_n\rangle$, we run on the l-th qubit
  1. Hadamard H and then,
  2. for $k > l$, R_{k-n+l} conditioned on the k-th qubit being $|1\rangle$.
- Why this procedure works (and the previous doesn't):
  Now the 2nd step replies only on the value of unaltered qubits!

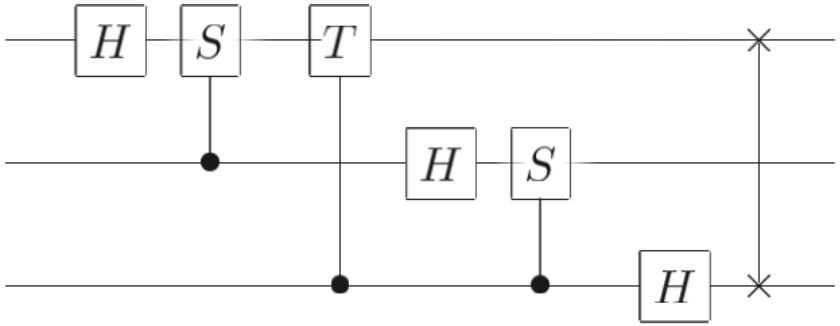
Circuit for QFT

- QFT circuit =



+ $\left\lfloor \frac{n}{2} \right\rfloor$ SWAP gates that reverse the order of qubits

- QFT for 3 qubits:



$S := T^2$

Quantum Hadamard transform vs QFT

- For a single qubit, Hadamard = QFT.
  QFT for a qubit: $|e_0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) = |+\rangle$, $|e_1\rangle := \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) = |-\rangle$.
  $\Rightarrow$ is just the Hadamard gate H!

- $|e_0\rangle = H^{\otimes n}|0\rangle^{\otimes n}$:
  To prepare $|e_0\rangle = 2^{-\frac{n}{2}}\sum_x |x\rangle$ for n qubits, it is enough to perform a Hadamard on each qubit, instead of executing the whole QFT.

Complexity of QFT

- Complexity of QFT:
  The n-qubit QFT has complexity $O(n^2)$.
- Comparison with the classical counterparts:
  The classical discrete Fourier transform on n-bits has complexity $O(4^n)$.
  A smarter implementation, fast Fourier transform, has complexity $O(n2^n)$.
- Conclusion: QFT is exponentially faster than its classical counterparts!
- Complexity of quantum Hadamard transform:
  The n-qubit quantum Hadamard transform has even lower complexity $O(n)$.
- We will see soon how to make use of this advantage!

Part III:
Quantum Phase Estimation

Quantum Phase Estimation (QPE) algorithm

- QPE outputs a good estimation of an eigenvalue of a unitary matrix U.
- Basic linear algebra:
  Any unitary U can be diagonalized as $U = \sum_l e^{2\pi i \cdot s_l} |u_l\rangle\langle u_l|$, where $s_l \in [0,1)$ and $\{|u_l\rangle\}$ is an ONB.
- Examples:
  Unitary $X = |+\rangle\langle +| + e^{i\pi}|-\rangle\langle -|$; eigenstates $|\pm\rangle$ with eigenvalues ± 1.
  Unitary $e^{i\frac{\theta}{2}X}$; eigenstates $|\pm\rangle$ with eigenvalue $e^{i\theta}$ and 1.

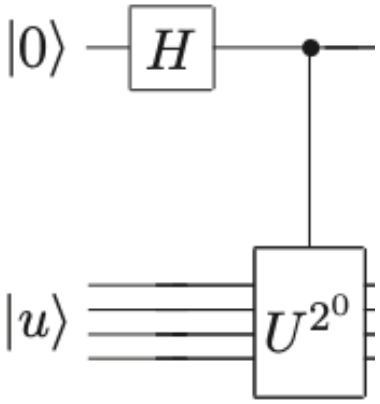
- QPE is a key subroutine in many quantum algorithms, including Shor and HHL (Lecture 5).

Overview of QPE

- Kitaev's Quantum Phase Estimation (QPE) algorithm:
  - Input: $2^t - 1$ access to the controlled version $C(U)$ of a gate U;
    an eigenstate $|u\rangle$, such that $U|u\rangle = e^{2\pi i \cdot \varphi}|u\rangle$,
  - Output: an estimate $\hat{\varphi}$ of φ, such that $|\hat{\varphi} - \varphi| < 2^{-n}$ with probability $1 - \epsilon$.
- Here the relation between the parameters is $t = n + \left\lceil \log\left(2 + \frac{1}{2\epsilon}\right) \right\rceil$.
- Example: $n = 5, \epsilon = 0.01 \Rightarrow$ QPE returns an estimate that is accurate to the 5th binary digit with probability 99%, requiring $2^{11} - 1 = 2047$ queries to $C(U)$
- Accuracy of QPE:
  Usually, we choose ϵ to be a small but fixed number and $n \approx t$.
  QPE achieves (in)accuracy δ ($|\hat{\varphi} - \varphi| < \delta$ almost surely) using $O(\log 1/\delta)$ qubits within $O(1/\delta)$ queries to U.

The phase kickback trick

- QPE uses the idea of "phase kickback", a trick that everyone should master!



- If $U|u\rangle = e^{2\pi i\varphi}|u\rangle$, we can apply $C(U)$ on $|+\rangle \otimes |u\rangle$:

$$C(U)(|+\rangle \otimes |u\rangle) = \frac{1}{\sqrt{2}}|0\rangle \otimes |u\rangle + \frac{1}{\sqrt{2}}|1\rangle \otimes (U|u\rangle)$$

$$= \frac{1}{\sqrt{2}}|0\rangle \otimes |u\rangle + \frac{1}{\sqrt{2}}|1\rangle \otimes (e^{2\pi i\varphi}|u\rangle)$$

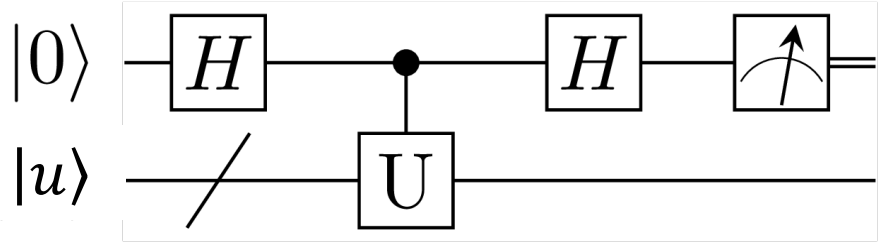
Phase kicked from 2nd to 1st qubit

$$= \frac{1}{\sqrt{2}}(|0\rangle + e^{2\pi i\varphi}|1\rangle) \otimes |u\rangle$$

2nd qubit decouples with the 1st qubit

1-qubit QPE

- With the phase kickback trick, we can do a “baby QPE” with one work qubit:

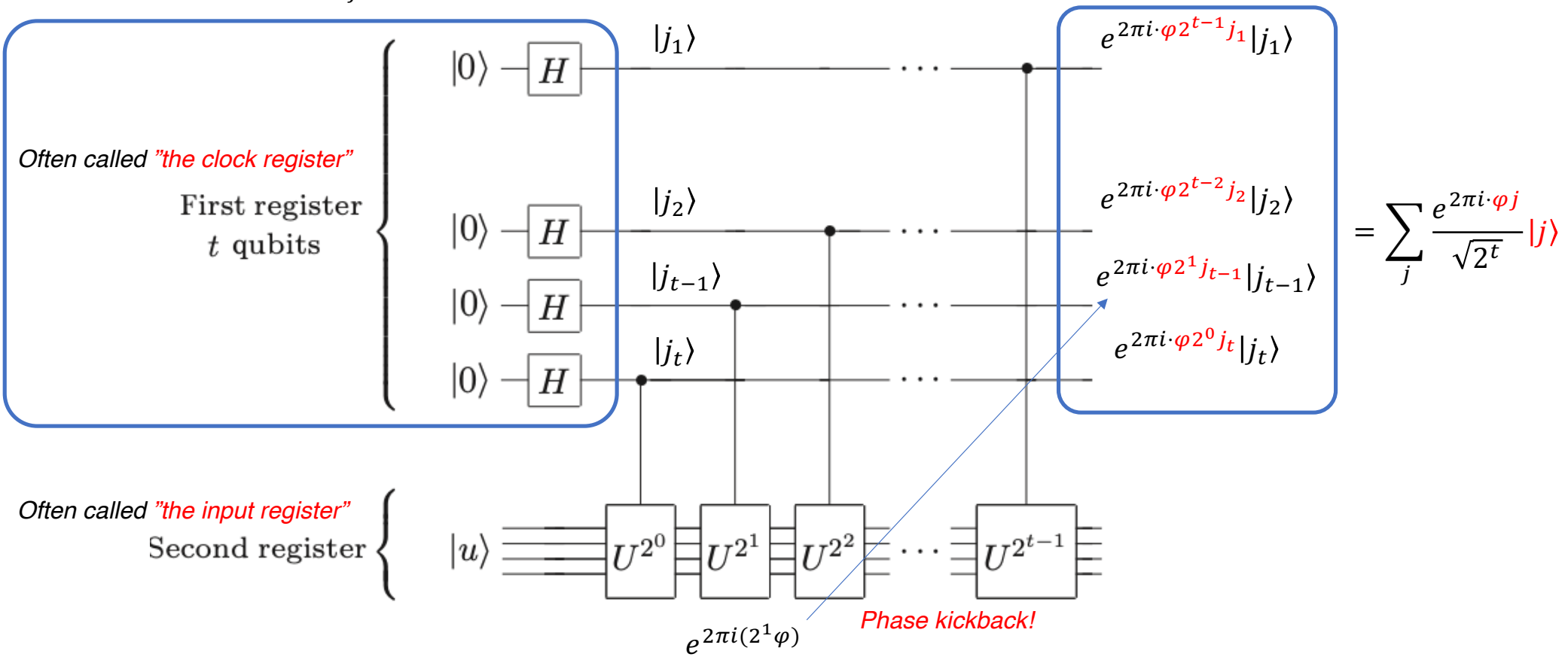


- Recall that $C(U)(|+\rangle \otimes |u\rangle) = \frac{1}{\sqrt{2}}(|0\rangle + e^{2\pi i\varphi}|1\rangle) \otimes |u\rangle$.
- The measurement: $P(0) = (\cos \pi\varphi)^2$ and $P(1) = (\sin \pi\varphi)^2$.
- When getting 0, we should conjecture that $P(0) > P(1)$, meaning that φ is closer to the boundary of the interval $[0,1]$.
- This estimation is very crude … but we can get higher precision by introducing more qubits!

Circuit of QPE

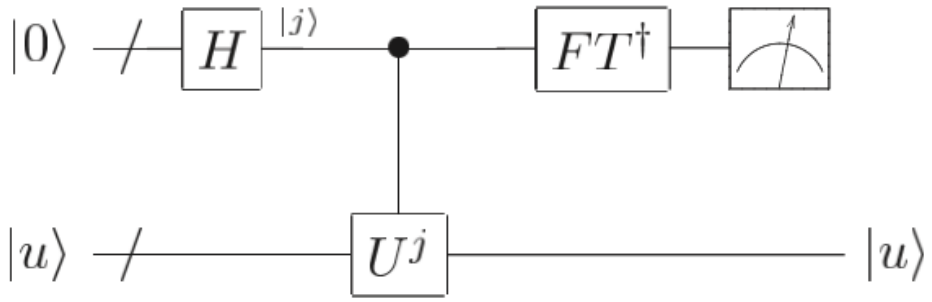
$$|e_0\rangle = \sum_j \frac{1}{\sqrt{2^t}}|j\rangle$$

$$j = 2^{t-1}j_1 + \cdots + j_t$$



$$= \sum_j \frac{e^{2\pi i\cdot\varphi j}}{\sqrt{2^t}}|j\rangle$$

Simpler representation of the QPE circuit

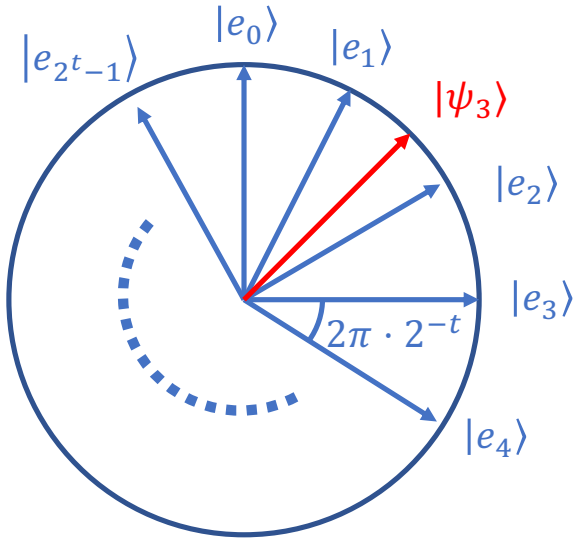


- Recall: we can write the computational basis $|j_1 \cdots j_t\rangle$ as $|j\rangle$ with $j = 2^{t-1}j_1 + \cdots + j_t$.
- Exercise:
  Show that the controlled gate part of QPE is equivalent to

$$O_{QPE} = \sum_{j=0}^{2^t-1} |j\rangle\langle j| \otimes U^j .$$

Accuracy of QPE

- Fourier basis as the “hand of a clock”:



- An ideal case:
  When $\varphi = 0.\varphi_1 \cdots \varphi_t$ without any approximation, for $j = 2^t \cdot \varphi$

$$p(j) = \left|\langle e_j|\psi_3\rangle\right|^2 = 1.$$

  The output estimate $\hat{\varphi}_t = \varphi$ does not have any error at all!

- In general, $\varphi = 0.\varphi_1 \cdots \varphi_t \varphi_{t+1} \cdots$ is a real number, and there will be an error.

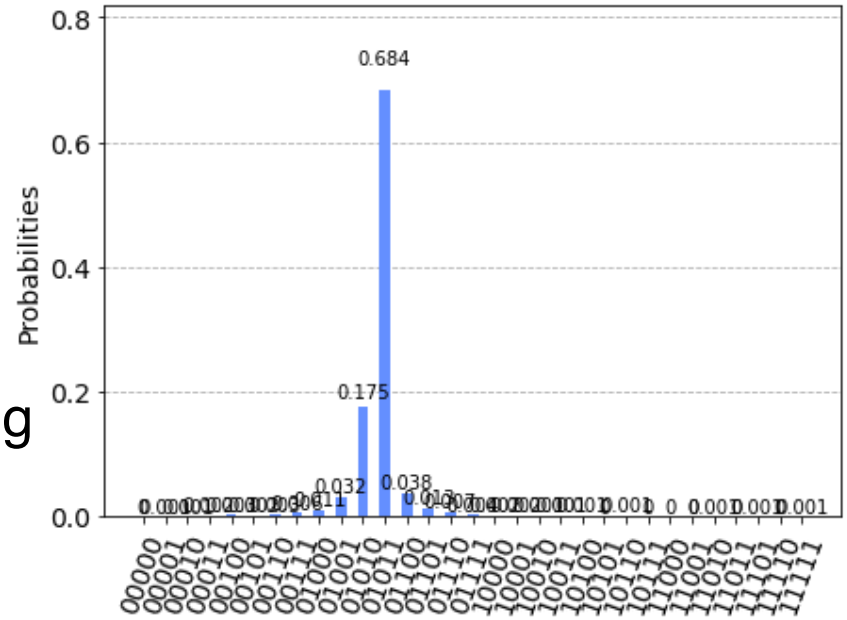
Accuracy of QPE

- In general, $\varphi \neq \hat{\varphi}_t$. How big is the error?

$$p(j) = \left| \sum_{k=0}^{2^t-1} \frac{e^{2\pi i \cdot k \cdot \delta\varphi_j}}{2^t} \right|^2 = \frac{1}{2^{2t}} \left(\frac{\sin(\pi \cdot 2^t \delta\varphi_j)}{\sin(\pi \cdot \delta\varphi_j)} \right)^2 \qquad \delta\varphi_j = \varphi - 0.j_1 \cdots j_t$$

$$\leq \frac{1}{2^{2t}} \left(\frac{1}{\sin(\pi \cdot \delta\varphi_j)} \right)^2 \leq \frac{1}{2^{2t}} \left(\frac{1}{\delta\varphi_j} \right)^2$$

- Probability goes down as $\delta\varphi_j$ (the error) increases.
- Therefore, by adding a few qubits (depending on ϵ) on top of n qubits, we can make the estimate accurate to the n-th digit.
- Try out QPE on OriginQ.



Output distribution of QPE ($t = 5$, $\varphi = 1/3$)

QPE Algorithm

- **Input:**
  1. Controlled unitary $C(U)$ (as an oracle).
  2. An eigenstate $|u\rangle$ of $C(U)$ with eigenvalue $e^{2\pi i\cdot\varphi}$
- **Output:**
  1. A t-bit estimate $\hat{\varphi}_t$ of φ
- **Procedure:**

1. Initialize the t-qubit main register and an ancilla in $|0\rangle_S|u\rangle_A$

$$|\psi_2\rangle = \frac{1}{\sqrt{2^t}}\sum_j |j\rangle \otimes |u\rangle$$

2. Perform Hadamard transform on S.

Phase kickback

3. For $k = 1, \dots, t$, apply 2^{t-k} queries of $C(U)$ on the k-th qubit (A as control).

$$|\psi_3\rangle = \sum_k \frac{e^{2\pi i\cdot\varphi k}}{\sqrt{2^t}}|k\rangle$$

4. Apply inverse QFT on S and measure in the computational basis of t qubits.

5. Output $\hat{\varphi}_t = 0.j_1j_2\cdots j_t$ when the measurement yields $j_1, j_2, \cdots, j_t$.

$$p(j) = |\langle e_j|\psi_3\rangle|^2$$

Exercise: Show that Step 4 realizes the measurement in the Fourier basis

Part IV: Shor's Algorithm

From QPE to factorization



"Mathematics is the Queen of the Sciences, and Number Theory the Queen of Mathematics"

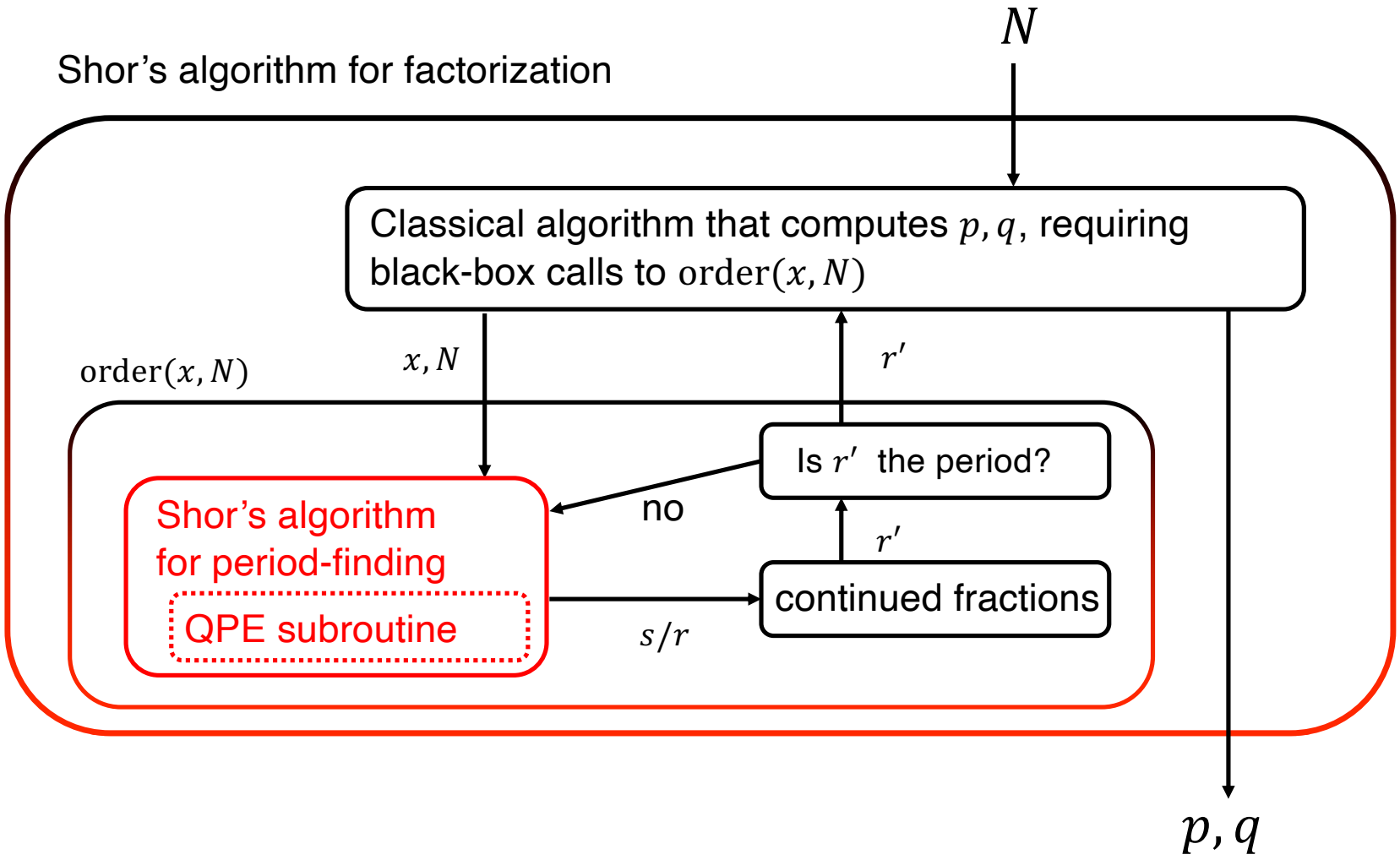
- Carl Friedrich Gauss

- Now we know how to measure the phase of a unitary using QPE.
- But our goal is to factorize a L-bit integer N.
- Consider another integer $x < N$ that is coprime with N. $L \coloneqq \lceil \log N \rceil$

The quantum part of Shor's algorithm finds the order of $x \bmod N$, which is the smallest integer $r > 0$ such that $f(r) = 1$ for $f(a) \coloneqq x^a \bmod N$.

- Why is this related to factorization?
- Idea (again, divide-and-conquer!):
  1. Reduce the factorization of $N = p \cdot q$ to the problem of finding the order of $x \bmod N$.
  2. Reduce the order-finding problem to a phase estimation problem.
  3. Use QPE algorithm to tackle the phase estimation!

What Shor's algorithm really does



Factorization, given order-finding subroutine

- **Input:**
  1) $N = p \cdot q$ for prime p, q and 2) the order-finding algorithm $\text{order}(x, N)$
- **Output:** $\{p, q\}$
- **Complexity:** $O((\log N)^3)$.
- **Procedure:**

Question:
What if one implies everything classically, including order-finding?

(the only quantum subroutine)
Evaluated with Shor's algorithm on a quantum computer!

Generate each digit randomly → $O(\log N)$

1. Check if $N = a^b$ (a is a factor if yes); randomly pick a positive integer x coprime with N.
2. Compute the order $r = \text{order}(x, N)$.

Exponentiation → $O(\text{polylog}\, N)$

3. If r is even (otherwise, go back to step 1) and $a + 1 \not\equiv 0 (mod\ N)$ for $a := x^{\frac{r}{2}} (mod\ N)$:

$$\{\gcd(a+1, N), \gcd(a-1, N)\}$$

contains at least one of p, q.

"greatest common divider" (e.g., gcd(6,10)=2), Euclidean → $O(\text{polylog}\, N)$

Why this works:

1. Note that $a^2 \equiv x^r (mod\ N) \equiv 1 (mod\ N)$ and thus $(a+1)(a-1) = s \cdot p \cdot q$ for some integer $s > 0$.
2. If $a - 1, a + 1 \not\equiv 0 (mod\ N)$, we must have either p or q "contained" in $a - 1$.

Order-finding, given QPE subroutine

- Suppose we have an oracle O for the modulo multiplication:

$$O\colon |y\rangle \mapsto |(xy) \pmod N\rangle$$

- Consider an ONB:

$$|u_s\rangle := \frac{1}{\sqrt{r}} \sum_{k=0}^{r-1} e^{2\pi i \cdot \left(-\frac{ks}{r}\right)} |x^k \pmod N\rangle$$

Exercise:
Prove that
1) $\{|u_s\rangle\}$ is indeed an ONB and
2) They are eigenstates of O.

- They are the eigenstates of O!

$$O|u_s\rangle = e^{2\pi i \cdot \frac{s}{r}} |u_s\rangle$$

- Idea: order finding = QPE + continued fractions (classical)
  Apply QPE to estimate the eigenvalues of $O \Rightarrow \varphi = \frac{s}{r}$ for some s.
  Then we can obtain r via continued fractions algorithm!

Continued fractions algorithm

- **Input:** Real number $\varphi > 0$.
- **Output:**
  1. The continued fractions $[a_0, a_1, \dots, a_M]$ such that

$$a_0 + \cfrac{1}{a_1 + \cfrac{1}{a_2 + \cfrac{1}{\dots + \cfrac{1}{a_M}}}} \approx \varphi$$

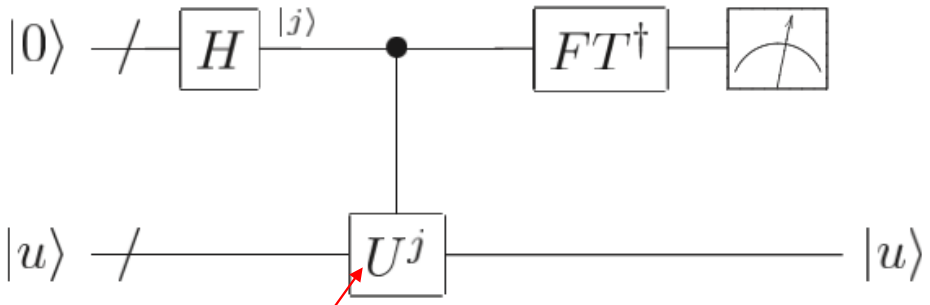
Recall: QPE achieves error threshold δ using $O(\log 1/\delta)$ qubits within $O(1/\delta)$ queries to U.

  2. From fractions one can obtain s/r where s, r are integers of L bits with $\left|\frac{s}{r} - \varphi\right| \leq \frac{1}{2r^2}$. This means that if φ is accurate to $\sim L$ digits, the output of continued fractions will be the correct r.

- It is enough to run QPE on t qubits; with $t \sim L = O(\log N)$.
- **Complexity of continued fractions**: $O(L^3)$ when r is L-bit $\Rightarrow O((\log N)^3)$

QPE for order-finding

- Now, the only task left is to construct a QPE for order-finding.
  Recall: QPE circuit



- That is, we need to (efficiently) implement the controlled version of
  $$O\colon |y\rangle \mapsto |(xy) \pmod N\rangle$$
  (whose eigenvalues depend on the order r) and its eigenstates $|u_s\rangle$.

- Question: How to construct $C(O)$ and $|u_s\rangle$ for some s?

Realization of the QPE oracle

- Instead of construct $O \to C(O) \to O_{QPE}$, we directly construct the QPE oracle over t qubits for the modular multiplication oracle O:

$$O_{QPE} = \sum_{j=0}^{2^t-1} |j\rangle\langle j| \otimes O^j$$

- It turns out that $O_{QPE} = O_{x,N}$, where $O_{x,N}$ is the oracle for $f(a) = x^a (mod\ N)$, acting on the system $|x\rangle$ and a $(\log N)$ - qubit ancilla $|q\rangle$

$$O_{x,N}: |a\rangle|q\rangle \mapsto |a\rangle|q \cdot x^a (\mathrm{mod}\ N)\rangle$$

- $O_{x,N}$ on t qubits can be realized with $O(t^3) = O((\log N)^3)$ gates (use modular exponentiation; cf. Box 5.2 of Nielsen & Chuang's book).

- Using $\left(\frac{1}{\sqrt{r}}\right)\sum_k O^j|u_k\rangle = |x^j (\mathrm{mod}\ \mathrm{N})\rangle$ (prove as an exercise) we can show that $O_{x,N}(|j\rangle|1\rangle) = |j\rangle|x^j (\mathrm{mod}\ N)\rangle$.

Eigenstates for QPE

- The rule of partial measurement, now applied to quantum order-finding.
- Problem: We cannot directly construct $|u_s\rangle$ since we don't know r.
- Property: $\left(\frac{1}{\sqrt{r}}\right)\sum_s |u_s\rangle = |1\rangle$ (Exercise: prove this property)
- Solution:
  It is enough to prepare $|1\rangle$, as it is a superposition of every element in $\{|u_k\rangle\}$.
- By Born's rule (for partial measurement), we get (an estimate of) s/r for $s = 0, \ldots, r-1$ with equal probability!
  (Yes, we don't know s, but continued fractions can still yield r!)

Continued fractions yield s and r with $\left|\frac{s}{r} - \varphi\right| \leq \frac{1}{2r^2}$

- Quantum order-finding = QPE on a superposition of different eigenvalues.

(brought over from L1) Partial measurement

- Born's rule for partial measurement:

$$P(x) = \langle\psi|(|\phi_x\rangle\langle\phi_x| \otimes I)|\psi\rangle$$

Bipartite state (e.g., the state of the clock register + input register in QPE)

Identity on the remaining part

- How should we interpret this?
  Suppose Alice and Bob each holds one part of the state.
  Alice measure in $\{|\phi_x\rangle\}_x$ and Bob measures in an arbitrary basis $\{|\phi'_y\rangle\}_y$

- Born's rule: joint probability of Alice getting x and Bob getting y

$$P(x,y) = |\langle\psi|(|\phi_x\rangle \otimes |\phi'_y\rangle)|^2 = \langle\psi|(|\phi_x\rangle\langle\phi_x| \otimes |\phi'_y\rangle\langle\phi'_y|)|\psi\rangle$$

- Alice's perspective:
  She does not know (or care) about Bob's outcome.
  Her local probability distribution:

Identity on the remaining part

$$P(x) = \sum_y P(x,y) = \langle\psi|\left(|\phi_x\rangle\langle\phi_x| \otimes \sum_y |\phi'_y\rangle\langle\phi'_y|\right)|\psi\rangle$$

Output of quantum order-finding

- We show that, in quantum order-finding, we get (an estimate of) s/r for $s = 0, \ldots, r-1$ with equal probability $1/r$.
- Since $\left(\frac{1}{\sqrt{r}}\right)\sum_s |u_s\rangle = |1\rangle$, the pre-measurement state is

$$|\psi\rangle = \left(\frac{1}{\sqrt{r}}\right)\sum_s |c_s\rangle \otimes |u_s\rangle$$

where $|c_s\rangle$ denotes the state of the clock register after O_{QPE} conditioned on the input register being $|u_s\rangle$.

Partial measurement: $P(e_x) = \langle\psi|(|e_x\rangle\langle e_x| \otimes I)|\psi\rangle$

- By Born's rule for partial measurement, the probability of getting e_x when measuring in the Fourier basis is:

$$P(e_x) = \frac{1}{r}\sum_s |\langle c_s|e_x\rangle|^2 = \frac{1}{r}\sum_s P(e_x|c_s)\,.$$

Probability of e_x when the state is $|c_s\rangle$.

Quantum order-finding

- **Input:**
  1. Oracle $O_{x,N}$: $|j\rangle|k\rangle \mapsto |j\rangle|\left(kx^j\right) \bmod N\rangle$.
- **Output:**
  1. (With high probability) the smallest $r > 0$ such that $x^r \equiv 1 \pmod N$
- **Query complexity**: $O(1)$ queries to $O_{x,N}$.

$L \coloneqq \lceil \log N \rceil$ is the length of N.

- **Procedure:**

1. Initialize the t-qubit main register and a L-qubit ancilla in $|0\rangle_S|1\rangle_A$
2. Perform Hadamard transform on S.
3. Apply $O_{x,N}$.
4. Apply inverse QFT on S and measure in the computational basis of t qubits.
   Return $\hat{\varphi}_t = 0.j_1 j_2 \cdots j_t$ when the measurement yields $j_1, j_2, \cdots, j_t$.
5. (Classical) Run continued fractions on $\hat{\varphi}_t \to s/r$.
   Check if the obtained r is the true order; restart from 1 if not.

A toy example: $N = 15$

1. Check if $N = a^b$ and output $\left(a, \frac{N}{a}\right)$ if so; otherwise proceed to 2.
2. Randomly pick $1 < x < N$.
3. Else if $\gcd(N, x) > 1$ (e.g., $x = 3$), factorization succeeds with output $\left(\gcd(N, x), \frac{N}{\gcd(N,x)}\right)$.
4. Else compute $r = \text{order}(x, N)$ with the order-finding subroutine:
   - If r is odd (e.g., $r = 3$ when $x = 8$) ❌, restart from Step 2.
   - If r is even (e.g., $r = 4$ when $x = 7$) ✅, compute $a = x^{\frac{r}{2}} (mod\ N)$ ($a = 4$ when $x = 7$)
   - If $a + 1 \not\equiv 0 (mod\ N)$ ✅ factorization succeeds with output $\{\gcd(a + 1, N), \gcd(a - 1, N)\}$.

Order-finding subroutine (snapshots for the case $x = 7$)

- Prepare the state $\frac{1}{\sqrt{2^t}}(|0\rangle + \cdots + |2^t - 1\rangle|1\rangle$ with $t = 11$
  (Exercise: discuss what happens if we choose other values of t)
- Apply oracle $|k\rangle|1\rangle \rightarrow |k\rangle\left|x^k (mod\ 15)\right\rangle$ with $x = 7$.
  The state becomes $\frac{1}{\sqrt{2^{11}}}(|0\rangle|1\rangle + |1\rangle|7\rangle + |2\rangle|4\rangle + |3\rangle|13\rangle + |4\rangle|1\rangle \cdots)$
  $= \frac{1}{\sqrt{2^{11}}}(|0\rangle + |4\rangle + \cdots) \otimes |1\rangle + (|1\rangle + |5\rangle + \cdots) \otimes |7\rangle + (|2\rangle + |6\rangle + \cdots) \otimes |4\rangle + (|3\rangle + |7\rangle + \cdots) \otimes |13\rangle$
- Measure the 2nd register in the computational basis.
  Suppose we get outcome "7" (other outcomes work in the same way).
  The state becomes $\frac{1}{\sqrt{2^9}}(|1\rangle + |5\rangle + |9\rangle + \cdots)$
- Measure the 1st register in the t-qubit Fourier basis:
  The non-zero probabilities are: $p(|e_j\rangle) = 1/4$ when j is a multiple of $\frac{2^t}{4} = 512$.
- For instance, if we get outcome $|e_j\rangle$ with $j = 1536$. Since $\frac{1536}{2^{11}} = \frac{1}{\left(1+\frac{1}{3}\right)} = \frac{3}{4}$, 4 is a candidate for the order, and we can verify that it is indeed the order.

Complexity of Shor's algorithm

- Shor's algorithm makes
  - One call to $N =^{?} a^b$.
  - $O(1)$ calls to $\gcd(x, N)$, $\mathrm{order}(x, N)$, and continued fractions.
- Complexity of $\mathrm{order}(x, N)$:
  The main overhead is the oracle $O_{x,N}$, which has complexity $O(L^3)$.
  $\mathrm{order}(x, N)$ makes only 1 query to $O_{x,N}$ and thus its complexity is $O(L^3)$.
- Complexity of $N =^{?} a^b$:
  (Omitted here for simplicity; cf. Exercise 5.17 of Nielsen & Chuang) also $O(L^3)$.
- Complexity of continued fractions, $\gcd(x, N)$:
  … also bounded by $O(L^3)$ (cf. Box 5.3 of Nielsen & Chuang) .
- Overall, the bottleneck of Shor is the complexity of implementing $O_{x,N}$.
- Conclusion: the complexity of Shor's algorithm is $O(L^3)$, with $L = \lceil \log N \rceil$.

Superiority of Shor's algorithm

- For factorization of a large number N.
- The complexity of Shor's algorithm is $O(\mathrm{polylog}(N))$.
- Existing classical algorithms sublinear in N:
  - Naïve way: enumerate 1 to $\sqrt{N} \rightarrow O(\sqrt{N})$.
  - Advanced ways (e.g., Pollard's ρ Algorithm): $O\left(N^{\frac{1}{4}}\right)$.
- **Conjectured exponential quantum advantage**.
  Shor's algorithm has an exponential speedup over existing classical algorithms. It is not proven (although likely) that there is no $O(\mathrm{polylog}(N))$ classical algorithms.
- If one can prove Factorization $\in$ NP, Shor would show BQP $>$ NP.

Long Journey to realizing Shor's theory …

10 years later…

Experimental realization of Shor's quantum factoring algorithm using nuclear magnetic r

Lieven M. K. Vandersypen*†, **Matthias Steffen***†
Costantino S. Yannoni*, **Mark H. Sherwood*** & I

* *IBM Almaden Research Center, San Jose, California*
† *Solid State and Photonics Laboratory, Stanford Uni*
California 94305-4075, USA

The number of steps any classical compute
find the prime factors of an *l*-digit integer
tially with *l*, at least using algorithms known
large integers is therefore conjectured to be i
an observation underlying the security of
graphic codes[1,2]. Quantum computers[3], h
integers in only polynomial time, using Sho
algorithm[4–6]. Although important for the
computers[7], experimental demonstration
proved elusive[8–10]. Here we report an im
simplest instance of Shor's algorithm: fac
(whose prime factors are 3 and 5). We use se

2001

Quantum Information Processing (2021) 20:112
https://doi.org/10.1007/s11128-021-03053-9

A hybrid scheme for prime factorization and its experimental implementation using IBM quantum processor

Ashwin Saxena[1] · **Abhishek Shukla**[1,2] · **Anirban Pathak**[1]

Received: 17 September 2020 / Accepted: 24 February 2021 / Published online: 17 March 2021
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2021

Abstract
We report a quantum–classical hybrid scheme for factorization of bi-prime numbers (which are odd and square-free) using IBM's quantum processors. The hybrid scheme proposed here involves both classical optimization techniques and adiabatic quantum optimization techniques, and is build by extending a previous scheme of hybrid factorization [Pal et al., Pramana 92, 26 (2019) and Xu et al., Phys. Rev. Lett. 108, 130501 (2012)]. The quantum part of the scheme is very general in the sense that it can be implemented using any quantum computing architecture. Here, as an example, we experimentally implement our scheme for prime factorization using IBM's QX4 quantum processor and have factorized 35.

20 years later…

nature photonics
LETTERS
PUBLISHED ONLINE: 21 OCTOBER 2012 | DOI: 10.1038/NPHOTON.2012.259

realization of Shor's quantum rithm using qubit recycling

hony Laing†, Thomas Lawson†, Roberto Alvarez, Xiao-Qi Zhou

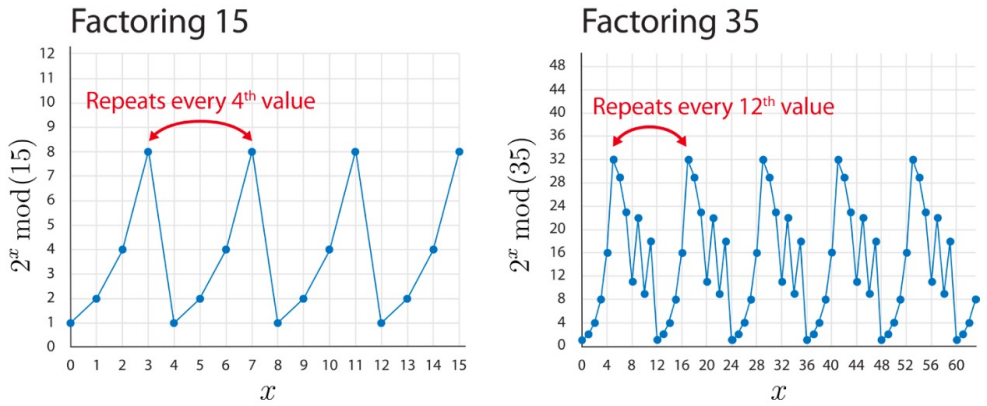


exploiting
splayed in
quantum
ransform-
sufficient
states of
ificant bit
nce of the
fact, the
t any one
aries con-
hose con-
that all
$(i+1)$th

How did it take so long?
(and RSA is still safe.)
Because quantum computers are much harder to scale up … (see later)

Quantum speedup of period finding

- A function has a period T if $f(T + a) = f(a)$ for every a.
- Shor's algorithm accelerates factorization, by finding the period of $f(a) \coloneqq x^a \bmod N$ in a quicker fashion.



- For any periodic function, a quantum computer achieves an acceleration in finding its period.
- In general, quantum computers work well with structured problems!

Summary

- Factorization and its importance in security.
- QFT & QPE.
- Shor’s algorithm of factorization.
- From QPE to factorization.

Homework

- Review the lecture slides; you may find the review questions in the next slides helpful.

  Try the exercises in the slides and discuss with your classmates.

- Arithmetic details like continued fractions are not important.

  Instead, you should understand how the quantum subroutine works and master the tricks of algorithm design (phase-kick, partial measurement …)

- Attempt Q1-4 in Assignment 2.

- Optional: Read p217-237 of *Quantum Computation and Quantum Information* by Nielsen and Chuang.

Review questions

- What is the phase kick-back trick?
  How is it used in QPE? What's the difference with the phase kick-back in the Deutsch-Jozsa algorithm?
- What are the differences and similarities between Hadamard transform and QFT?
- For $\varphi \in [0,1)$, define the d - dim quantum clock state in as $|\psi_\varphi\rangle := \sum_{k=0}^{d-1} e^{2\pi i \cdot j\varphi}|j\rangle$. Express the Fourier basis as special instances of the clock states.
- Compare the performances of two strategies: 1) Do 1-qubit QPE and repeat many times, and 2) Use all queries in one-shot of (t-qubit) QPE.
- When are two quantum clock states $|\psi_\varphi\rangle, |\psi_{\varphi'}\rangle$ orthogonal to each other (i.e., to have zero fidelity)?